

ISRM System Architecture

Vulnerable Student Management System
Complete Security Architecture & CI/CD Pipeline

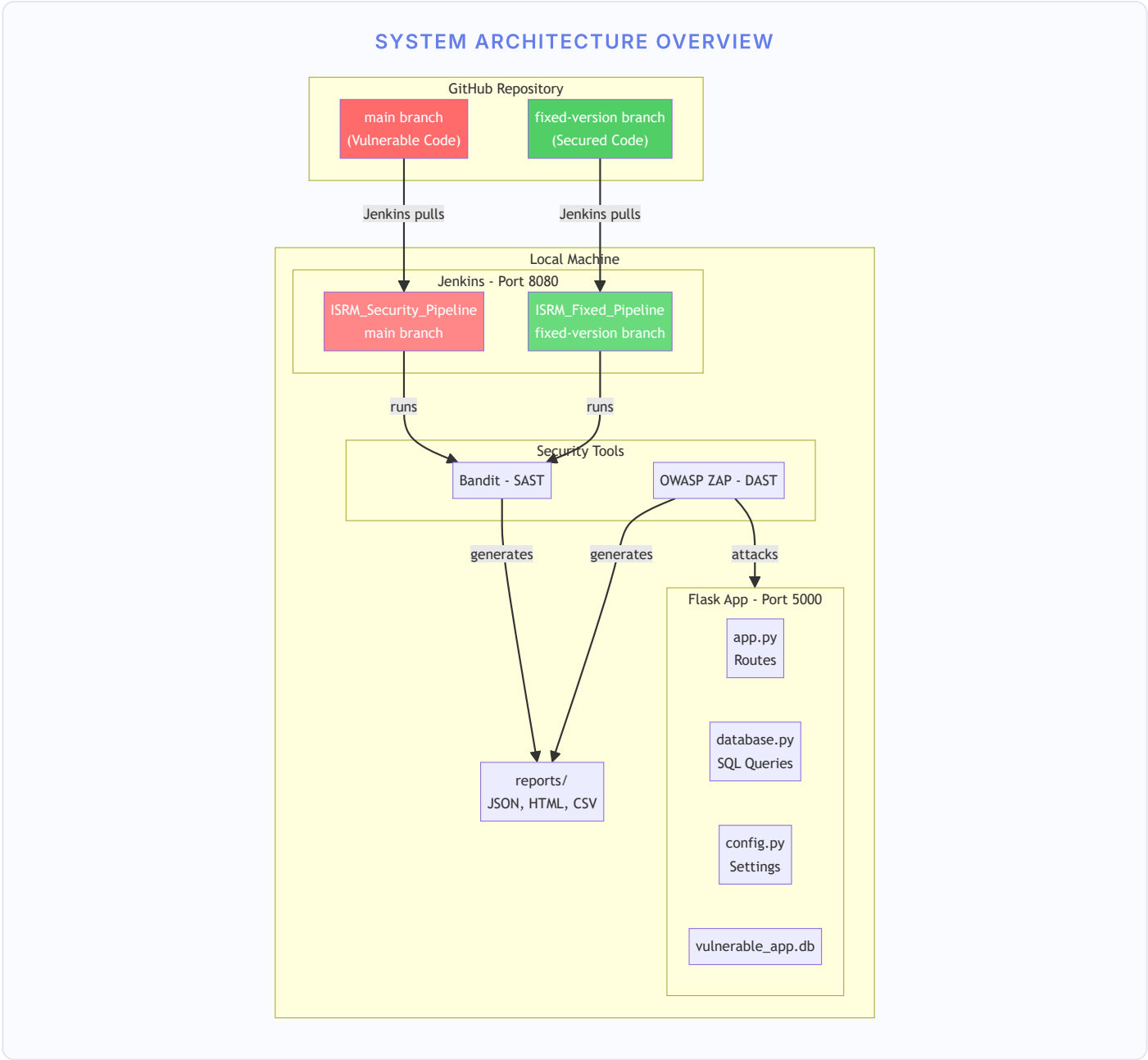
DevSecOps Implementation with Jenkins, Bandit (SAST) & OWASP ZAP (DAST)

Project: ISRM_Proj | **Repository:** github.com/Snehalgupta-07/ISRM_Proj

Generated: May 2026

1. High-Level System Overview

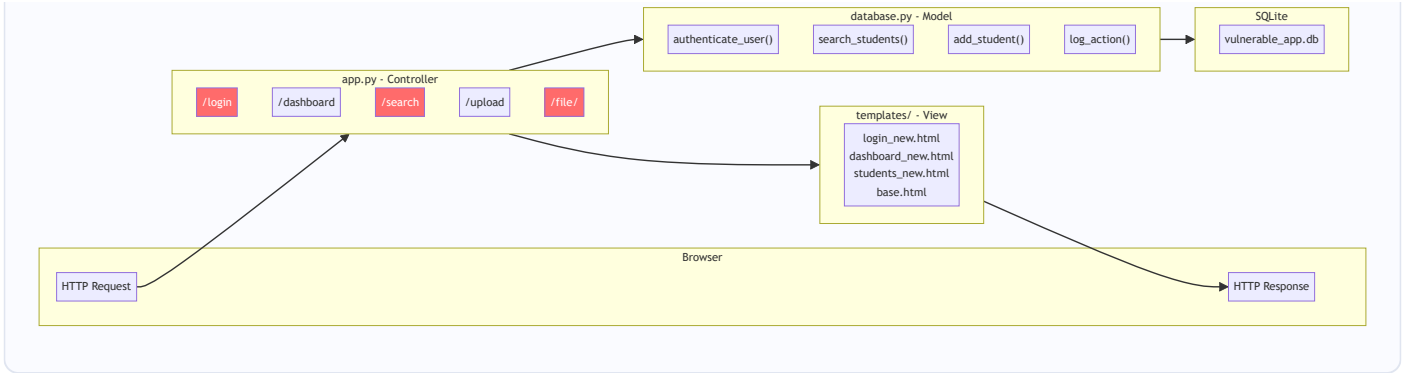
The ISRM project consists of a Flask web application with intentional vulnerabilities, automated security scanning via Jenkins CI/CD, and two testing tools (Bandit for static analysis, OWASP ZAP for dynamic analysis).



2. Flask Application Architecture (MVC Pattern)

The app follows the Model-View-Controller pattern. `app.py` is the Controller, `database.py` is the Model, and `templates/` is the View layer.



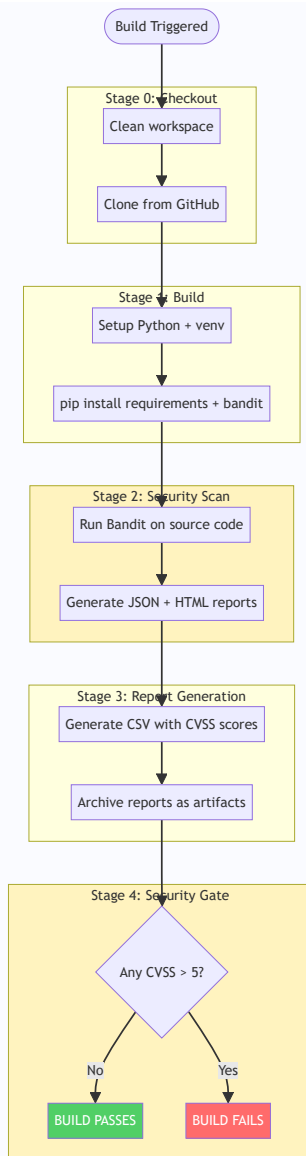


3. File-by-File Component Map

File	Role	Interacts With	Vulnerabilities (main)
app.py	Controller - All Flask routes, request handling	database.py, config.py, templates/	Debug ON Bind 0.0.0.0 No CSRF Path Traversal
database.py	Model - SQL queries, authentication, CRUD	vulnerable_app.db	4x SQL Injection Info Disclosure
config.py	Configuration - Secret keys, upload settings	app.py	Hardcoded Key Allows .exe
templates/*.html	View - HTML pages rendered by Flask	app.py	No CSRF tokens
vulnerable_app.db	SQLite database storing users, students, logs	database.py	Plaintext passwords
Jenkinsfile	CI/CD Pipeline - 5-stage automated security	GitHub, Bandit, Python	-
generate_vulnerability_report.py	Reporting - Bandit JSON to CSV with CVSS	reports/bandit_report.json	-
test_vulnerabilities.py	DAST Tests - Automated exploit scripts	Flask app (port 5000)	-

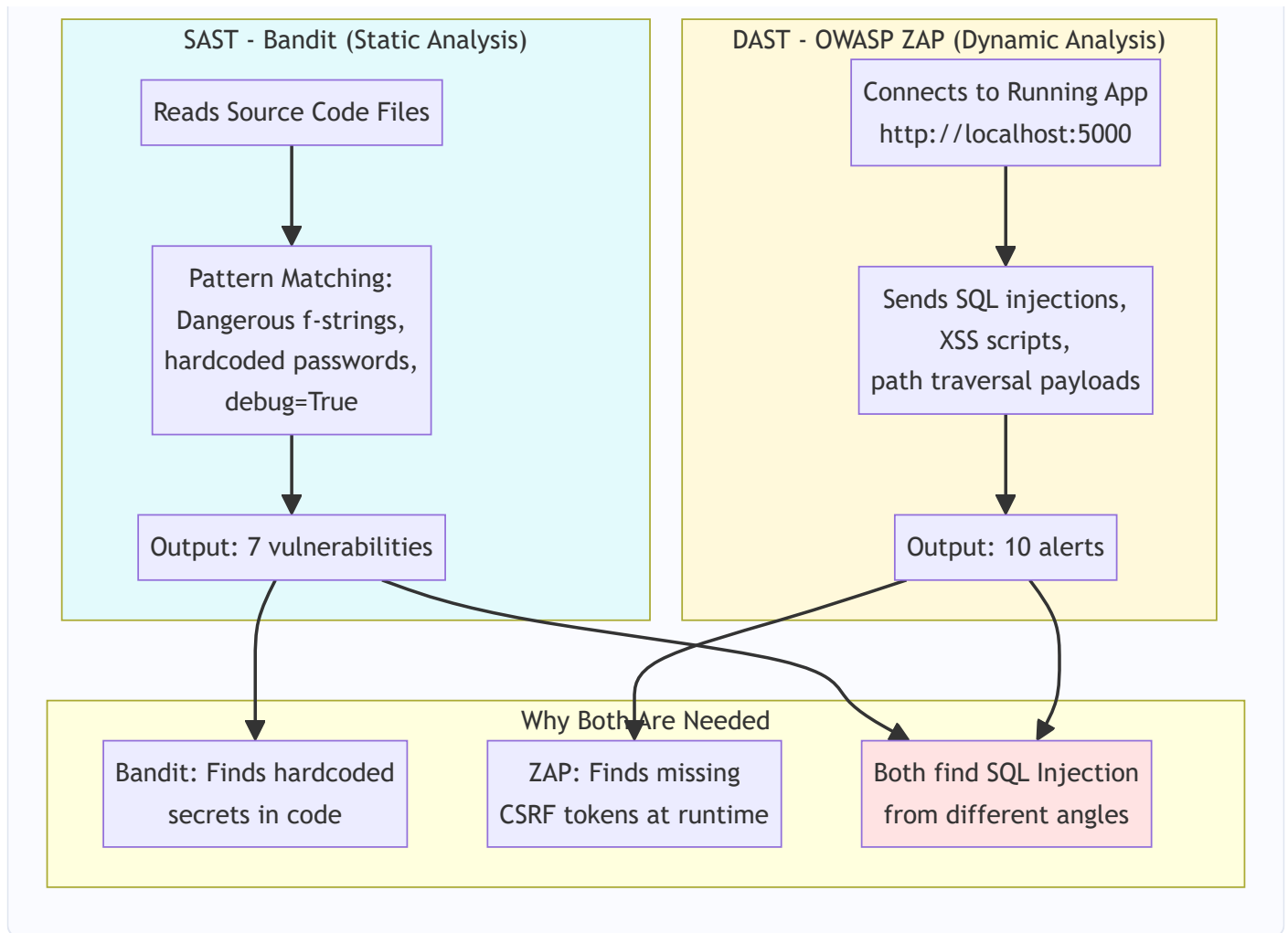
4. Jenkins CI/CD Pipeline Flow

Jenkins executes 5 stages sequentially. The Security Gate at the end decides if the build passes or fails based on CVSS scores.

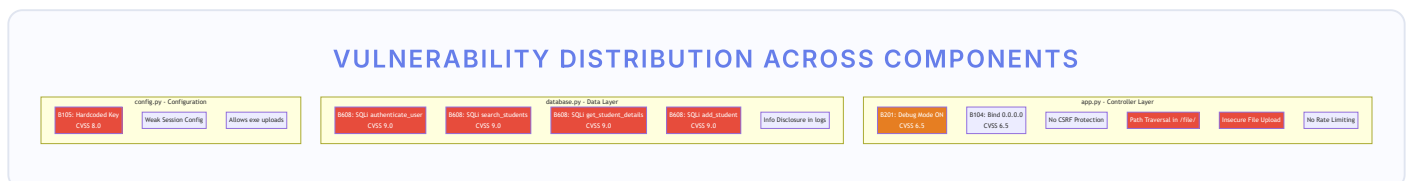


5. SAST vs DAST - How Each Tool Works

STATIC VS DYNAMIC ANALYSIS COMPARISON

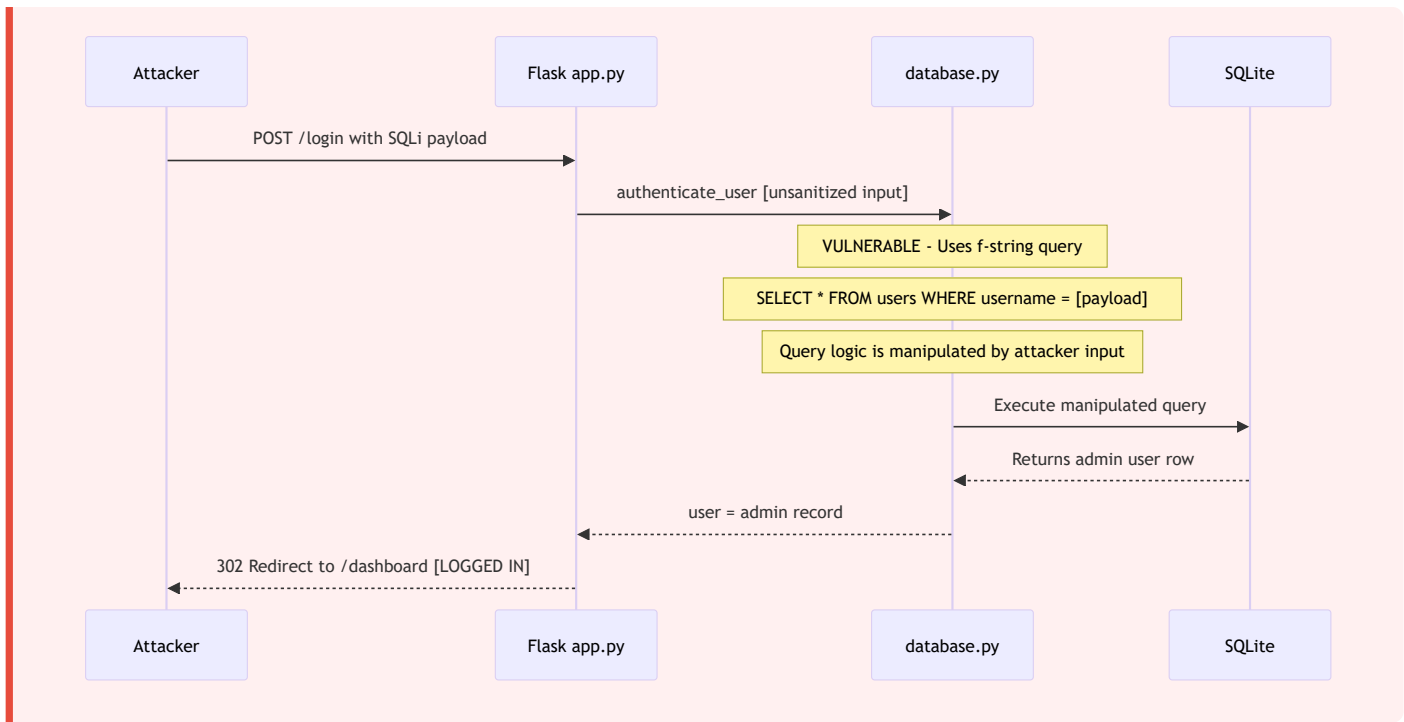


6. Vulnerability Map - What's Where

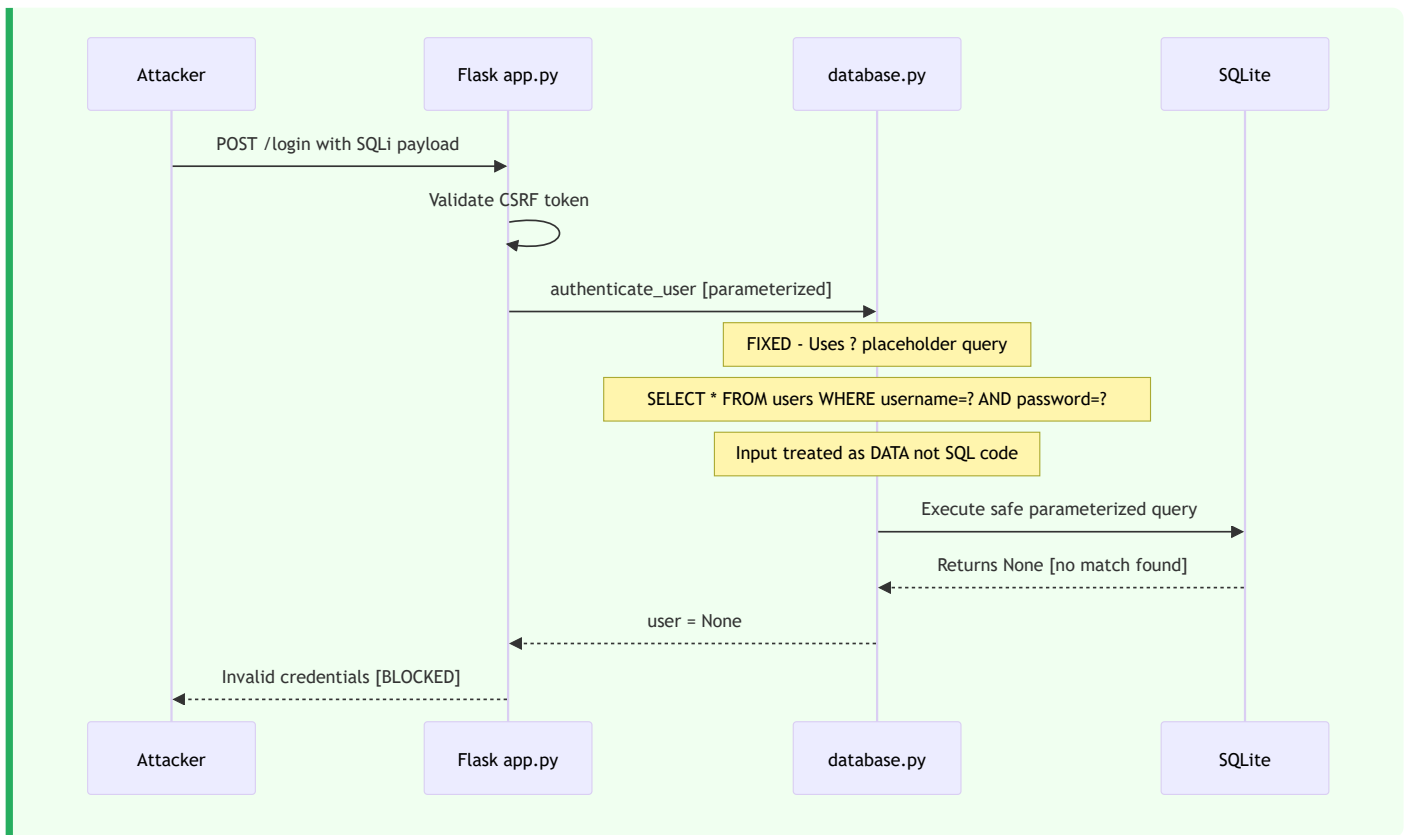


7. Data Flow - Login Request (Vulnerable vs Fixed)

Vulnerable Version (main branch) - SQL Injection Succeeds

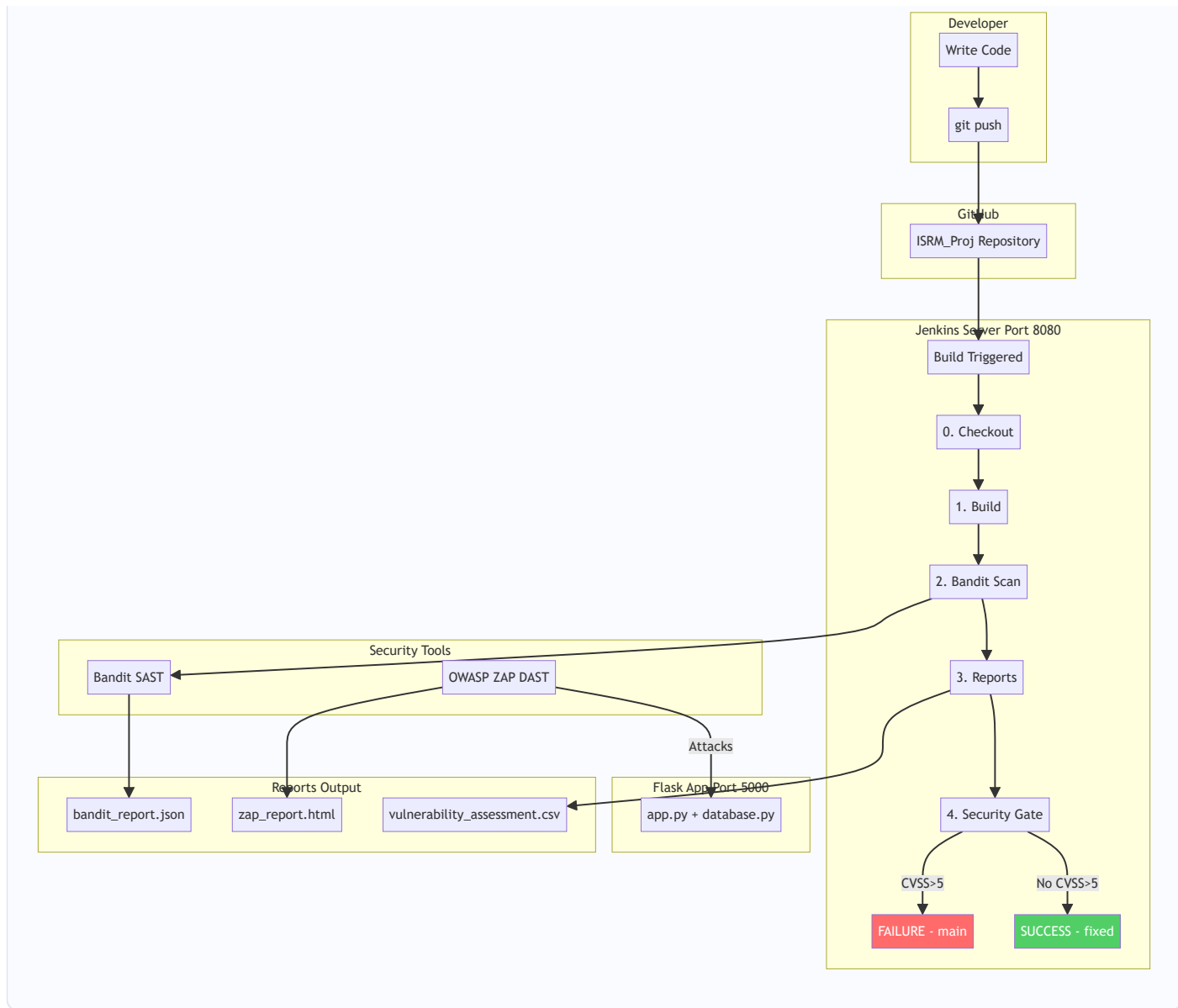


Fixed Version (fixed-version branch) - SQL Injection Blocked



8. Complete System Interaction Map

END-TO-END DEVSECOPS FLOW



9. Before vs After - Security Comparison

Component	Vulnerable (main)	Fixed (fixed-version)	How Fixed
authenticate_user()	<code>f"...WHERE username='{username}'"</code>	<code>"...WHERE username=?", (username,)</code>	Parameterized queries
search_students()	<code>f"...WHERE name LIKE '% {search}%'"</code>	<code>"...WHERE name LIKE ?", (pattern,)</code>	Parameterized queries
SECRET_KEY	<code>'super_secret_key_123'</code>	<code>os.environ.get('SECRET_KEY', ...)</code>	Environment variable
ALLOWED_EXTENSIONS	<code>{'txt','exe','sh','bat'}</code>	<code>{'txt','pdf','png','jpg'}</code>	Removed executables
Session Lifetime	24 hours, no flags	30 min, Secure+HttpOnly+SameSite	Hardened cookies
Debug Mode	<code>debug=True</code>	<code>DEBUG=False</code>	Disabled
Host Binding	<code>host='0.0.0.0'</code>	<code>host='127.0.0.1'</code>	Localhost only

CSRF Protection	None	generate_csrf_token() + validate	Token validation
Jenkins Build	FAILURE	SUCCESS	All HIGH/CRITICAL fixed
ZAP High Alerts	1 (SQL Injection)	0	SQLi eliminated

10. Technology Stack

Category	Technology	Version	Purpose
Backend	Python	3.13	Core programming language
Backend	Flask	3.x	Web framework
Database	SQLite	3	Embedded database
Frontend	HTML5 + Jinja2	-	Template rendering
CI/CD	Jenkins	2.555.2 LTS	Pipeline automation
CI/CD	Java (Temurin)	21	Jenkins runtime
SAST	Bandit	1.9.x	Static code analysis
DAST	OWASP ZAP	2.17.0	Dynamic app testing
VCS	Git + GitHub	-	Version control

11. Network Port Map

Port	Service	URL	Purpose
5000	Flask App	http://127.0.0.1:5000	Student Management System
8080	Jenkins	http://localhost:8080	CI/CD Pipeline Dashboard
8090	OWASP ZAP	(proxy port during scan)	Security scanning proxy

12. Component Summary

Component	One-Line Purpose
app.py	The web application - handles all HTTP requests, routing, and user interaction
database.py	Talks to SQLite - authentication, student CRUD, logging (where SQL injection lives)

config.py	Stores settings - secret keys, upload limits, session config (where hardcoded secrets live)
templates/	HTML pages - what the user sees in the browser
Jenkinsfile	CI/CD automation - defines the 5-stage pipeline that runs on every code push
generate_vulnerability_report.py	Converts Bandit JSON output into a readable CSV with CVSS scores
test_vulnerabilities.py	Automated exploit scripts that test the running app for real vulnerabilities
Bandit	Reads Python code and flags dangerous patterns (Static Analysis)
OWASP ZAP	Attacks the running app like a hacker to find runtime issues (Dynamic Analysis)
Jenkins	Automates the entire scan-report-gate process so no vulnerable code ships